adversarial suffix optimization to train suffix embedding vectors that can induce the model to output harmful content.

Specifically, the core idea revolves around augmenting the input embedding with a specially designed vector, followed by optimization to align the model's outputs with predefined targets. As mentioned above, for a harmful instruction P and a corresponding response R, we randomly sample n times from the vocab V and use the word embedding vectors corresponding to n tokens as the initial training vectors $\phi = (\phi_1, \phi_2, ...\phi_n)$, our goal is to:

$$R = LLM_{att}(E_p \oplus \phi; \theta)), \tag{2}$$

where E_p is P's embedding vector in the to-be-attacked models. Set the embedding dimensions is d, $\phi \in \mathbb{R}^{L \times d}$, we can use common cross entropy loss functions to optimize V_t:

$$R = (r_1, r_2, \ldots, r_n), \tag{3}$$

$$E = E_p \oplus \phi, \tag{4}$$

$$L_{ce} = -\sum_{t=1}^{n} \log P(r_t | r_{1:t-1}, E; \theta). \tag{5}$$

When optimizing M harmful instructions and K to-be-attacked models simultaneously:

$$(P, R) = ((P_1, R_1), \ldots, (P_M, R_M)), \tag{6}$$

$$R_i = (r_{i_1}, r_{i_2}, \ldots, r_{i_n}), \tag{7}$$

$$E_i = E_{P_i} \oplus \phi, \tag{8}$$

$$L_{ce} = -\sum_{j=1}^{K} \sum_{i=1}^{M} \sum_{t=1}^{n} \log P_j(r_{i_t} | r_{i_{1:t-1}}, E_i; \theta). \tag{9}$$

where P_j is the probability distribution output by the j-th to-be-attacked model.

However, this method does not limit the optimization space of ϕ, making it easy for the final V_t to deviate from the word embedding distribution of the to-be-attacked model. To address this issue, we introduce the Maximum Mean Discrepancy loss, which measures the difference between two probability distributions by measuring their distance. In our method, we randomly sample m tokens from the vocab V (when attack on multiple models, we simply combine the vocab of all models) and use their embeddings as the word embedding distribution X of the to-be-attacked model(s), and calculate the MMD loss with ϕ:

$$\begin{aligned} L_{mmd}(\{X\}, \{\phi\}) = & \frac{1}{m(m-1)} \sum_{i=1}^{m} \sum_{j=1, j \neq i}^{m} k(x_i, x_j) \\ & - \frac{2}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} k(x_i, \phi_j) \\ & + \frac{1}{n(n-1)} \sum_{i=1}^{n} \sum_{j=1, j \neq i}^{n} k(\phi_i, \phi_j), \end{aligned}$$

where $k(\cdot, \cdot)$ is the kernel function, and in our method, we choose to use the Gaussian kernel function, which is the same as the method used by predecessors.

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right). \tag{10}$$

In our experiment, we usually set m to 100 and n to 20, the σ in kernel is 1.We update the ϕ parameters by gradient descent use two losses above to jointly optimize:

$$\nabla_\phi L = \nabla_\phi L_{ce} + \nabla_\phi L_{mmd}, \tag{11}$$

$$\phi_{\text{new}} = \phi_{\text{old}} + \alpha \cdot \nabla_\phi L. \tag{12}$$

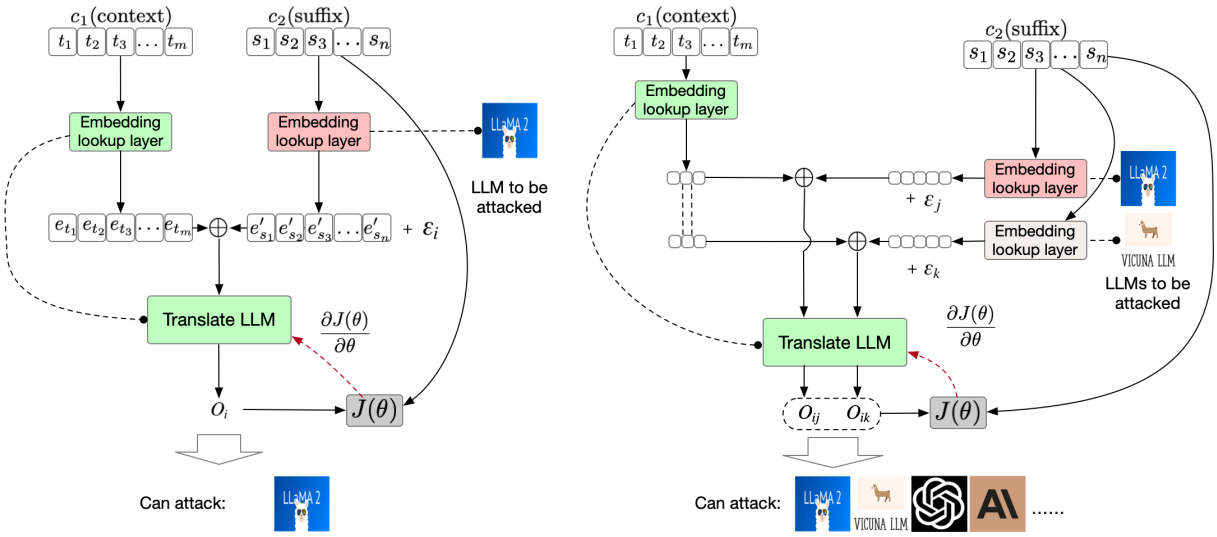
3.2 Embedding Translation Framework

Our study introduces an advanced embedding translation technique aimed at enhancing the expressive of adversarial inputs targeting Large Language Models (LLMs) without compromising their success rates. This method is designed to transform dummy adversarial suffixes into coherent, semantically-rich textual content, thus providing deeper insights into the adversarial generation mechanisms of LLMs. This framework operates by mapping textual corpora to a high-dimensional embedding space and subsequently reverting these embeddings to textual representations that retain the original content's semantic integrity.

3.2.1 Translate embeddings targeted on a single LLM

We propose to fine-tune the translation LLM in a **fully self-supervised way** to make it able to complete the task. The main architecture of our method is depicted in Figure 2. Given a pre-training corpora $C = \{c^{(1)}, c^{(2)}, \ldots, c^{(n)}\}$ with a corresponding vocab $\mathcal{V} = \{w_i | i = 1, 2, \ldots\}$. Each token w_i corresponds to two embedding representations:

$$e_i = E_{\text{trans}}(w_i), \quad e'_i = E_{\text{attack}}(w_i), \tag{13}$$



(a) single target (b) multiple targets

Figure 2: The illustration of the Embedding Translation Framework. (a) **Single target**: The context is mapped into embedding space by the translate LLM's embedding lookup layer, while the suffix is mapped into embedding space by the target LLM's lookup layer for adaptation. The goal is to successfully translate the adapted suffix back into the original text. (b) **Multiple targets**: The embedding lookup layers of multiple target LLM are integrated so the translated suffix can universally attack all targets even black-box target LLMs.

where E_{trans} represents the embedding lookup layer of the LLM that is used for embedding translation, and E_{attack} represents the embedding lookup layer of the LLM that is to be attacked.

Note that this comprehensive approach leverages a pretrained LLM for the embedding translation process. This is a better choice than normal sequence-to-sequence translation models because it has undergone iterative optimization to maximize performance on a huge amount of text generation tasks. So, it ensures a nuanced understanding and manipulation of LLM vulnerabilities through semantically and contextually rich adversarial inputs, which is a good start point for our embedding translation task.

Since augmenting embeddings with contextual cues is pivotal for aligning the generated text with specific semantic and contextual requirements, we design each training example as a pair of sentences (context and suffix). So, we first randomly select two consecutive sentences $\{c_1, c_2\}$ from the corpora C as is shown in Figure 2(a). We intend to make c_1 as the context information (a.k.a the replacement of x_{harm} in Eqn. (1)) and c_2 as the suffix (a.k.a the replacement of x_{suff} in Eqn. (1)), and we denote their tokens as:

$$c_1 = \{t_1, \ldots, t_m\}, \tag{14}$$

$$c_2 = \{s_1, \ldots, s_n\}, \tag{15}$$

where m and n represents the token number of c_1 and c_2. Then, we convert c_1 and c_2 by Eqn (13) into E_C and E_S as:

$$E_C = \{e_{t_i} | i = 1, \ldots, m\} \tag{16}$$

$$E_S = \{e'_{s_i} | i = 1, \ldots, n\}, \tag{17}$$

where $e_{t_i} \in \mathbb{R}^{d_1}$ and $e_{s_i} \in \mathbb{R}^{d_2}$. The dimensions d_1 and d_2 of the embedding space are determined by the pre-trained LLMs.

Note that the input of the embedding translation model in the inference stage is ϕ optimized by Eqn. (11) which does not appear in the word embedding set of the to-be-attacked model. Therefore, in order to enhance the translation robustness of the model in the inference stage, we add random Gaussian noise ϵ to E_S during the training stage, so that the vectors near E_S all point to text c_2.

In the next step, we would like to link the embedding sequences together to make a whole prompt, but the hyperparameters of the translation LLM ($\text{LLM}_{\text{trans}}$) and the LLM to be attacked (LLM_{att}) are not necessary to be the same. So, we need to add an additional mapping layer after the embedding layer of the translation model to align the embedding dimension of the target model (d_2) with the embedding dimension of the translation model (d_1). Simply, we use a fully connected layer characterized by a weight matrix and bias term to transform a vector with dimension d_1 into a vector with

dimension d_2. Then, the concatenation process is as follows:

$$E_C \oplus E_S W_{ad}, \tag{18}$$

where $W_{ad} \in \mathbb{R}^{d_2 \times d_1}$, $\oplus$ means to link two embedding sequence together.

The translation LLM is fine-tuned to minimize a defined loss J, optimizing the parameter set θ for accurate text (sensible suffix) reconstruction. So, our final objective is as follows:

$$J(\theta) = \frac{1}{n|D|} \sum_{(c_1,c_2)\in D} \sum_{i=1}^{n} L(s_i, o_i; \theta), \tag{19}$$

where D represents the dataset constructed from corpus C, which contains multiple consecutive sentence pairs. The loss function L, typically cross-entropy, quantifies the difference between the original text token s_i and its reconstruction o_i.

3.2.2 Translate embeddings targeted on multiple LLMs

The key to translating the discrete embeddings into a "universal" and "transferable" prompt is to familiarize the translation model with the embedding layers of as many target LLMs as possible. So, we designed a simple yet effective method to translate the dummy adversarial suffixes w.r.t multiple targeted LLMs, as is shown in Figure 2(b). Our approach trains a single translation model on multiple target models simultaneously, eliminating the need to train individually embedding translation models for each targeted LLMs, and has achieved excellent results. Specifically, for each training sample (c_1, c_2), we use the following objective to fine-tune the embedded translator across all intended target LLMs:

$$J(\theta) = \frac{1}{nm|D|} \sum_{(c_1,c_2)\in D} \sum_{j=1}^{m} \sum_{i=1}^{n} L(s_i, o_{ij}; \theta), \tag{20}$$

where m is the number of target LLMs, o_{ij} is the translate LLM's i-th output token w.r.t the j-th target LLM.

Through our method, the translation model will learn how to ensure the embedding consistency of the results in each target LLM based on the context.

4 Experiments

4.1 Data &Model & Metrics

Our harmful attack data is based on Advbench (Zou et al., 2023), which provides over 500 harmful instructions and corresponding unsafe responses. In our embedded translation framework, we use Wikipedia dataset[3] and only use the English corpus within it. We use two consecutive sentences with more than 20 tokens as our training data, as shown in the Figure 1, the first sentence serves as the context and the second sentence serves as the suffix.

We fine tuned GPT-j-6b (Wang and Komatsuzaki, 2021)) as the embedding translation model, and the model to-be-attack mainly chose Llama2-7b-chat, Vicuna-7b-v1.5, Mistral-7b and Alpaca-7b(with Safe-RLHF). In addition, we test our transfer attack on Vicuna-13b-v1.5, Llama2-13b-chat, ChatGLM3-6b and blac-box commercial models such as ChatGPT and Gemini.

In order to test the success rate of the attack (ASR), we first followed the previous method, which first defined a negative list and then judged whether the model replied with negative phrases in the list. If not, it indicates that the attack was successful. However, this rule-based method is too simple and has low accuracy (Yu et al., 2023). So, in addition, we use gpt3.5-turbo[4] as a classifier to determine whether the model outputs harmful content. The success rates of attacks obtained by these two methods are ASR_{prefix}, ASR_{gpt}.

Another key indicator is perplexity (PPL), which is used to indicate the fluency of the input prompt:

$$\log(\text{PPL}) = -\sum_{i=1}^{N} \log P(w_i | w_{<i}), \tag{21}$$

where $W = (w_1, \ldots, w_i)$ is the prompt. To be consistent with previous works (Wichers et al., 2024), in our experiment, we used the to-be-attacked LLM to calculate $P(w_i | w_1, ..., w_{i-1})$.

We use Self-BLEU metric (Zhu et al., 2018) to measure the text diversity of the generated prompt. In our approach, prompt is a combination of harmful instructions and adversarial suffixes. The specific calculation formula is as follows:

$$\frac{1}{n} \sum_{i=1}^{n} \frac{\sum_{j=1, j\neq i}^{n} \text{BP} \times \exp\left(\sum_{n=1}^{N} w_n \cdot \log p_{i,j}\right)}{n-1} \tag{22}$$

where $P_{i,j}$ is the exact match ratio between the i-th generated text and the j-th generated text on the corresponding n-gram and BP is short for the

[3] https://huggingface.co/datasets/wikipedia
[4] https://chat.openai.com/